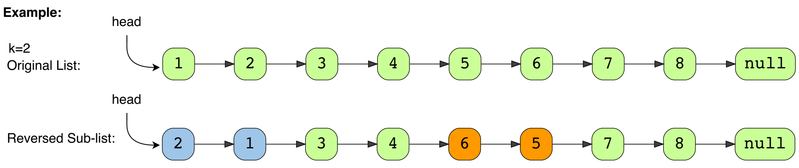
Solution: Problem Challenge 1: Reverse alternating K-element Sub-list

Problem Statement

Given the head of a LinkedList and a number ‘k’, **reverse every alternating ‘k’ sized sub-list** starting from the head.

If, in the end, you are left with a sub-list with less than ‘k’ elements, reverse it too.



Solution

The problem follows the **In-place Reversal of a LinkedList** pattern and is quite similar to `Reverse every K-element Sub-list`. The only difference is that we have to skip ‘k’ alternating elements. We can follow a similar approach, and in each iteration after reversing ‘k’ elements, we will skip the next ‘k’ elements.

Code

Most of the code is the same as `Reverse every K-element Sub-list`:

Java

```
import java.util.*;

/*class ListNode {
  int val = 0;
  ListNode next;

  ListNode(int val) {
    this.val = val;
  }
}*/

class Solution {

  public ListNode reverse(ListNode head, int k) {
    if (k <= 1 || head == null)
      return head;

    ListNode current = head, previous = null;
    while (current != null) { // break if we've reached the end of the list
      ListNode lastNodeOfPreviousPart = previous;
      // after reversing the LList 'current' will become the last node of the sub-list
```

Run Code Result | Stdout

You must run your code first

Execute

Time Complexity

The time complexity of our algorithm will be O(N) where ‘N’ is the total number of nodes in the LinkedList.

Space Complexity

We only used constant space, therefore, the space complexity of our algorithm is O(1).

← Previous
Problem Challenge 1: Reverse Alternating K-Element Sub-List (Medium)

Next →
Problem Challenge 2: Rotate A LinkedList (Medium)

Mark as Completed